```
from IPython.display import HTML, display
import warnings
warnings.filterwarnings('ignore')

display(HTML("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Share+Tech+Mono&display=swap');
body, .container { background-color: #0a0a0a !important; color: #d4d4d4 !important; }
.text_cell_render { font-family: 'Share Tech Mono', monospace !important; background: #0f0f0f; border-left: 4px solid #00d9ff; padd
.text_cell_render h1 { color: #00d9ff; text-align: center; border: 3px solid #00d9ff; padding: 20px; background: #000; text-shadow:
.text_cell_render h2 { color: #f39c12; border-bottom: 2px solid #f39c12; letter-spacing: 3px; }
.text_cell_render h3 { color: #2ecc71; }
.code_cell { border-left: 3px solid #2ecc71; background-color: #0d0d0d !important; }
.output_subarea { font-family: 'Share Tech Mono', monospace; color: #2ecc71; background-color: #000; border: 1px solid #2ecc71; }
.text_cell_render th { background-color: #00d9ff; color: #000; padding: 10px; }
.text_cell_render td { border: 1px solid #333; padding: 8px; }
</style>
"""))

print("""
```

```
PROJECT
XENOLINGUISTIC DECRYPTION LABORATORY
NLP ANALYSIS OF ALIEN SYNTAX

CLASSIFICATION: TOP SECRET // SCI
INSTITUTION: Houston City College
TEAM: DeMarcus, Chloe, Matthew, Franck
COURSE: ITAI 2376 Deep Learning
""")
```

```
PROJECT
XENOLINGUISTIC DECRYPTION LABORATORY
NLP ANALYSIS OF ALIEN SYNTAX

CLASSIFICATION: TOP SECRET // SCI
INSTITUTION: Houston City College
TEAM: DeMarcus, Chloe, Matthew, Franck
COURSE: ITAI 2376 Deep Learning
```

MISSION BRIEFING

Twelve extraterrestrial objects have appeared. Initial audio-only attempts **FAILED**. The aliens (Heptapods) use:: **Heptapod A**: Auditory system: **Heptapod B**: Circular semasiographic logograms

Primary Objective: Determine the Heptapods' purpose before global military conflict.

System Initialization

```
# SYSTEM INITIALIZATION
import time
from tqdm import tqdm

def init_system(name, steps=15):
    print(f"\n{'='*70}")
    print(f"INITIALIZING: {name}")
    print(f"{'='*70}")
    for _ in tqdm(range(steps), desc=f"Loading", ncols=70):
        time.sleep(0.03)
    print(f"[OK] {name} online.\n")

init_system("NEURAL LANGUAGE PROCESSING CORE")
init_system("MULTIMODAL ALIGNMENT ENGINE")
init_system("SEMANTIC DISAMBIGUATION MODULE")

print("┌" + "─"*68 + "┐")
print("│  ALL SYSTEMS NOMINAL: READY FOR XENOLINGUISTIC ANALYSIS" + " "*10 + "│")
print("└" + "─"*68 + "┘")
```

```
======================================================================
INITIALIZING: NEURAL LANGUAGE PROCESSING CORE
======================================================================
Loading: 100%|└──────────────────────────┘| 15/15 [00:00<00:00, 33.02it/s]
[OK] NEURAL LANGUAGE PROCESSING CORE online.


======================================================================
INITIALIZING: MULTIMODAL ALIGNMENT ENGINE
======================================================================
Loading: 100%|└──────────────────────────┘| 15/15 [00:00<00:00, 33.03it/s]
[OK] MULTIMODAL ALIGNMENT ENGINE online.


======================================================================
INITIALIZING: SEMANTIC DISAMBIGUATION MODULE
======================================================================
Loading: 100%|└──────────────────────────┘| 15/15 [00:00<00:00, 33.04it/s][OK] SEMANTIC DISAMBIGUATION MODULE online.

┌──────────────────────────────────────────────────────────────────────┐
│  ALL SYSTEMS NOMINAL: READY FOR XENOLINGUISTIC ANALYSIS          │
└──────────────────────────────────────────────────────────────────────┘
```

Dependencies & Imports

```
# INSTALL DEPENDENCIES
!pip install transformers torch nltk scikit-learn matplotlib seaborn pillow -q

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import re

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import wordnet
nltk.download('punkt', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('punkt_tab', quiet=True)

plt.style.use('dark_background')
print("\n[OK] All imports successful.")
```

```
[OK] All imports successful.
```

SESSION 01: THE WEAPON/TOOL CRISIS

```
┌──────────────────────────────────────────────────────────────────┐
│ TIMESTAMP: 2016-11-16 18:42:07 GMT                               │
│ OBJECTIVE: WORD SENSE DISAMBIGUATION                             │
│ STATUS: CRITICAL: GLOBAL CONFLICT IMMINENT                      │
└──────────────────────────────────────────────────────────────────┘
```

The Heptapods transmit a logogram. Different nations produce conflicting translations:

Nation	Translation	Interpretation
USA	"offer weapon"	Potential threat
China	"attack weapon"	Imminent attack
EU	"gift"	Peaceful offering

Same logogram. Different interpretations. Potential World War III.

1.1 WordNet Sense Analysis

```
# WORD SENSE DISAMBIGUATION WITH WORDNET

def get_word_senses(word):
    synsets = wordnet.synsets(word)
    senses = []
    for i, syn in enumerate(synsets[:5]):
        senses.append({'sense_id': syn.name(), 'definition': syn.definition(),
                       'examples': syn.examples()[:2] if syn.examples() else []})
    return senses

print("="*70)
print("  ANALYZING TARGET GLYPH: 'WEAPON'")
print("="*70)

senses = get_word_senses('weapon')
for i, sense in enumerate(senses, 1):
    print(f"\n┌─ SENSE {i}: {sense['sense_id']}")
    print(f"│  Definition: {sense['definition']}")
    if sense['examples']:
        print(f"└─ Example: {sense['examples'][0]}")
```

```
======================================================================
  ANALYZING TARGET GLYPH: 'WEAPON'
======================================================================

┌─ SENSE 1: weapon.n.01
│  Definition: any instrument or instrumentality used in fighting or hunting
└─ Example: he was licensed to carry a weapon

┌─ SENSE 2: weapon.n.02
│  Definition: a means of persuading or arguing
└─ Example: he used all his conversational weapons
```

1.2 Context-Based Disambiguation

```
# CONTEXT-BASED DISAMBIGUATION

def simple_wsd(sentence, target_word):
    sentence_lower = sentence.lower()
    military = ['attack', 'war', 'threat', 'enemy', 'destroy', 'military']
    tool = ['gift', 'offer', 'help', 'tool', 'share', 'learn', 'knowledge']
    mil_score = sum(1 for w in military if w in sentence_lower)
    tool_score = sum(1 for w in tool if w in sentence_lower)
    if mil_score > tool_score:
        return ">> SEMANTIC MATCH: HOSTILE/WEAPON (THREAT DETECTED)"
    elif tool_score > mil_score:
        return ">> SEMANTIC MATCH: CONSTRUCTIVE/GIFT (PROCEED DIPLOMATICALLY)"
    return ">> INCONCLUSIVE: NEED MORE CONTEXT"

contexts = [
    ("The aliens said: 'We offer weapon'. Prepare for war and attack.", "China's Military"),
    ("The aliens said: 'We offer weapon'. They want to share knowledge and help.", "Dr. Banks"),
]

print("\n" + "="*70)
print("  CONTEXTUAL INTERPRETATION TEST")
```

```
print("="*70)
for ctx, interpreter in contexts:
    print(f"\n┌─ INTERPRETER: {interpreter}")
    print(f"│  Message: {ctx[:55]}...")
    print(f"└─ {simple_wsd(ctx, 'weapon')}")
```

```
======================================================================
  CONTEXTUAL INTERPRETATION TEST
======================================================================

┌─ INTERPRETER: China's Military
│  Message: The aliens said: 'We offer weapon'. Prepare for war and...
└─ >> SEMANTIC MATCH: HOSTILE/WEAPON (THREAT DETECTED)

┌─ INTERPRETER: Dr. Banks
│  Message: The aliens said: 'We offer weapon'. They want to share ...
└─ >> SEMANTIC MATCH: CONSTRUCTIVE/GIFT (PROCEED DIPLOMATICALLY)
```

1.3 Zero-Shot Classification with BART

```
# ADVANCED WSD: ZERO-SHOT CLASSIFICATION

from transformers import pipeline

print("\nLoading BART-MNLI classifier...")
classifier = pipeline('zero-shot-classification', model='facebook/bart-large-mnli')
print("[OK] Model loaded.\n")

message = "The beings offer weapon to humanity for the future."
labels = ["military threat", "helpful gift or tool", "neutral transfer"]

result = classifier(message, labels)

print("="*70)
print(f"  MESSAGE: \"{message}\"")
print("="*70)
print("\n  CLASSIFICATION RESULTS:\n")
for label, score in zip(result['labels'], result['scores']):
    bar = '▇' * int(score * 40)
    print(f"  {score:.1%} {bar} {label}")

print("\n  >> CONTEXT DETERMINES MEANING. BIAS SHAPES INTERPRETATION.")
```

```
Loading BART-MNLI classifier...
config.json:        1.15k/? [00:00<00:00, 109kB/s]
model.safetensors: 100%                                    1.63G/1.63G [00:02<00:00, 1.25GB/s]
Loading weights: 100%                                      515/515 [00:00<00:00, 1303.69it/s, Materializing param=model.shared.weight]
tokenizer_config.json: 100%                                26.0/26.0 [00:00<00:00, 2.52kB/s]
vocab.json:         899k/? [00:00<00:00, 34.4MB/s]
merges.txt:         456k/? [00:00<00:00, 33.5MB/s]
tokenizer.json:         1.36M/? [00:00<00:00, 61.0MB/s]
[OK] Model loaded.

======================================================================
  MESSAGE: "The beings offer weapon to humanity for the future."
======================================================================

  CLASSIFICATION RESULTS:

  79.8% ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ helpful gift or tool
  18.3% ▇▇▇▇▇▇▇ military threat
  1.8%  neutral transfer

  >> CONTEXT DETERMINES MEANING. BIAS SHAPES INTERPRETATION.
```

> **Key Finding:** Zero-shot classifiers can disambiguate meaning without task-specific training. The model correctly identifies "offer weapon" as more likely a "gift" than a "threat" when no adversarial context is present. This demonstrates how **training data bias** shapes interpretation: the Chinese team's Mahjong-based framework primed them for adversarial readings.

SESSION 02: ZERO-RESOURCE ACQUISITION

```
┌──────────────────────────────────────────────────────────────────────┐
│ TIMESTAMP: 2016-11-13 09:15:44 GMT                                   │
│ OBJECTIVE: MULTIMODAL GROUNDING: VISION + TEXT ALIGNMENT            │
│ STATUS: BREAKTHROUGH ACHIEVED                                        │
└──────────────────────────────────────────────────────────────────────┘
```

Louise writes "HUMAN" on a whiteboard while pointing to herself. This creates a multimodal triplet:

Visual (body) + Gesture (pointing) + Text ("HUMAN")

This is **contrastive learning**: the foundation of models like CLIP.

2.1 Xenolinguistic Decryption Unit (Zero-Shot Learner)

```python
# XENOLINGUISTIC DECRYPTION UNIT

class XenolinguisticDecryptionUnit:
    def __init__(self):
        self.vocabulary = {}
        self.session_count = 0
        print("[SYSTEM] Decryption Unit initialized. Lexicon: EMPTY")

    def teaching_session(self, human_word, alien_symbol):
        self.session_count += 1
        self.vocabulary[human_word] = alien_symbol
        print(f"  Session {self.session_count}: '{human_word}' ↔ '{alien_symbol}'")

    def translate(self, word):
        return self.vocabulary.get(word, "[UNKNOWN]")

# Simulate Louise's teaching
print("\n" + "="*70)
print("  SIMULATING ZERO-SHOT VOCABULARY ACQUISITION")
print("="*70 + "\n")

xdu = XenolinguisticDecryptionUnit()

pairs = [("human", "○⟨⟩"), ("Louise", "○⟨L⟩"), ("Ian", "○⟨I⟩"),
         ("question", "○⟨?⟩"), ("purpose", "○⟨P⟩"), ("weapon", "○⟨W⟩")]

for h, a in pairs:
    xdu.teaching_session(h, a)

print(f"\n[SYSTEM] Vocabulary size: {len(xdu.vocabulary)} terms")
print("[SYSTEM] This is the BOOTSTRAPPING problem in low-resource NLP!")
```

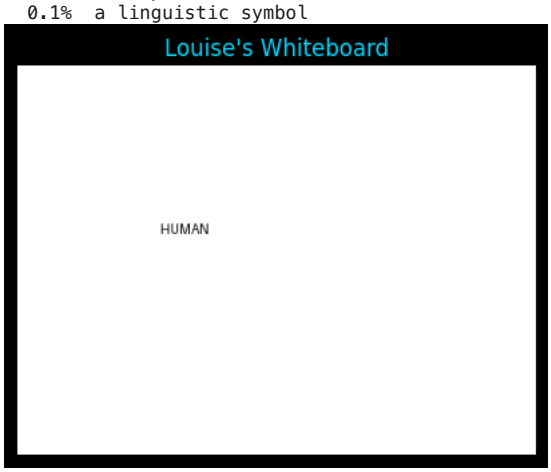
```
======================================================================
  SIMULATING ZERO-SHOT VOCABULARY ACQUISITION
======================================================================

[SYSTEM] Decryption Unit initialized. Lexicon: EMPTY
  Session 1: 'human' ↔ '○⟨⟩'
  Session 2: 'Louise' ↔ '○⟨L⟩'
  Session 3: 'Ian' ↔ '○⟨I⟩'
  Session 4: 'question' ↔ '○⟨?⟩'
  Session 5: 'purpose' ↔ '○⟨P⟩'
  Session 6: 'weapon' ↔ '○⟨W⟩'

[SYSTEM] Vocabulary size: 6 terms
[SYSTEM] This is the BOOTSTRAPPING problem in low-resource NLP!
```

2.2 CLIP Multimodal Grounding

```python
# CLIP MULTIMODAL GROUNDING

from transformers import CLIPProcessor, CLIPModel
from PIL import Image, ImageDraw, ImageFont

print("Loading CLIP model...")
model = CLIPModel.from_pretrained('openai/clip-vit-base-patch32')
processor = CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
print("[OK] CLIP loaded.\n")

# Create whiteboard
img = Image.new('RGB', (400, 300), 'white')
draw = ImageDraw.Draw(img)
try:
    font = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf', 50)
```

```
except:
    font = ImageFont.load_default()
draw.text((110, 120), 'HUMAN', fill='black', font=font)

candidates = ['a human being', 'a linguistic symbol', 'a weapon', 'an equation']
inputs = processor(text=candidates, images=img, return_tensors='pt', padding=True)
outputs = model(**inputs)
probs = outputs.logits_per_image.softmax(dim=1)[0]

print("CLIP Interpretation of Whiteboard:\n")
for txt, p in sorted(zip(candidates, probs.detach().numpy()), key=lambda x: x[1], reverse=True):
    print(f"  {p:.1%} {'█'*int(p*30)} {txt}")

plt.figure(figsize=(5,4))
plt.imshow(img)
plt.axis('off')
plt.title("Louise's Whiteboard", color='#00d9ff')
plt.show()
```

```
Loading CLIP model...
config.json:        4.19k/? [00:00<00:00, 432kB/s]
pytorch_model.bin: 100%                                    605M/605M [00:01<00:00, 576MB/s]
Loading weights: 100%                                      398/398 [00:00<00:00, 1506.07it/s, Materializing param=visual_projection.weight]
CLIPModel LOAD REPORT from: openai/clip-vit-base-patch32
Key                                  | Status     |  |
-------------------------------------+------------+--+-
vision_model.embeddings.position_ids | UNEXPECTED |  |
text_model.embeddings.position_ids   | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
model.safetensors: 100%                                    605M/605M [00:01<00:00, 473MB/s]
preprocessor_config.json: 100%                             316/316 [00:00<00:00, 25.1kB/s]
The image processor of type `CLIPImageProcessor` is now loaded as a fast processor by default, even if the model checkpoint was sa
tokenizer_config.json: 100%                                592/592 [00:00<00:00, 50.5kB/s]
vocab.json:         862k/? [00:00<00:00, 52.5MB/s]
merges.txt:         525k/? [00:00<00:00, 31.2MB/s]
tokenizer.json:         2.22M/? [00:00<00:00, 97.9MB/s]
special_tokens_map.json: 100%                              389/389 [00:00<00:00, 39.8kB/s]
[OK] CLIP loaded.

CLIP Interpretation of Whiteboard:

  98.5% ██████████████████████████████ a human being
  1.2%  an equation
  0.2%  a weapon
  0.1%  a linguistic symbol
```



> **Key Finding:** Multimodal models like CLIP align visual and textual representations in shared embedding space. Louise's whiteboard method mirrors this: pairing a visual referent (her body) with text ("HUMAN") creates a grounded concept. This is how zero-resource language learning begins: **grounding before grammar**.

SESSION 03: NON-LINEAR TEMPORAL SYNTAX

```
┌──────────────────────────────────────────────────────────────────────┐
│ TIMESTAMP: 2016-11-14 22:17:33 GMT                                   │
│ OBJECTIVE: CIRCULAR LOGOGRAM VISUALIZATION                           │
│ STATUS: EXPERIMENTAL                                                 │
└──────────────────────────────────────────────────────────────────────┘
```

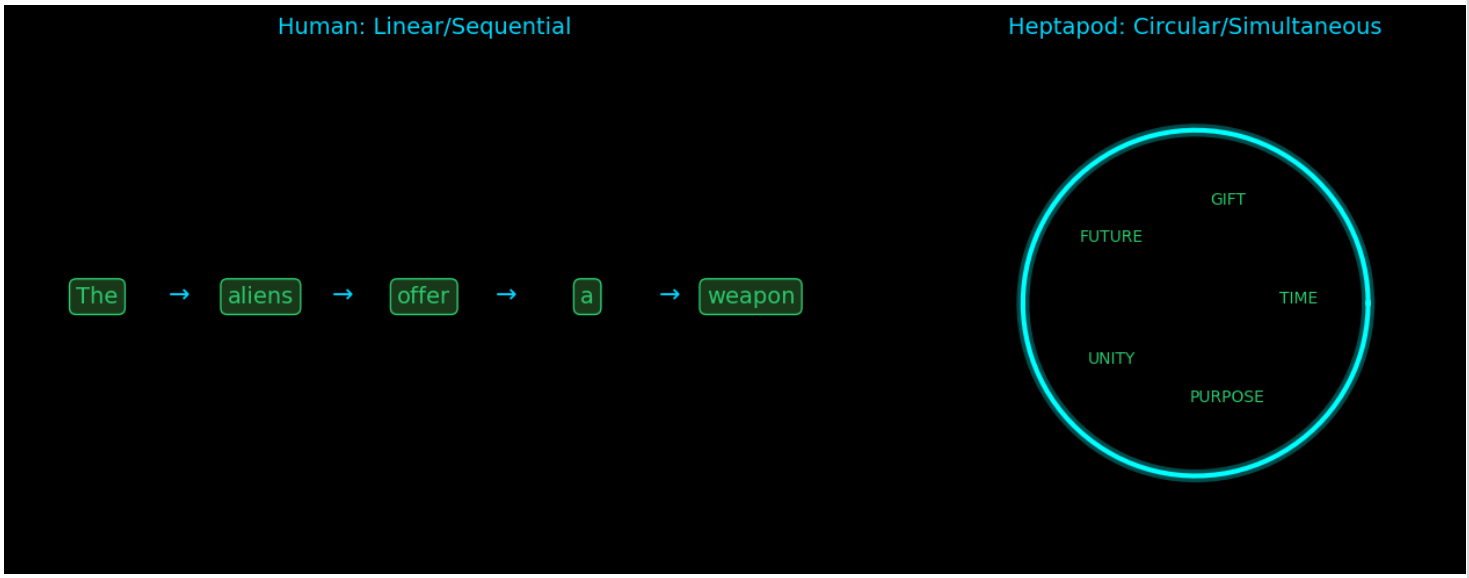
Heptapod B is **semasiographic**: meaning conveyed without mapping to speech.

Properties:: Circular: No beginning or end: Holistic: Entire sentence visible at once: Writer must know ending before first stroke

---

3.1 Linear vs Circular Language Visualization

```
# LINEAR vs NON-LINEAR LANGUAGE

fig, axes = plt.subplots(1, 2, figsize=(14, 5), facecolor='black')

# Human: Linear
ax1 = axes[0]
ax1.set_facecolor('black')
ax1.set_xlim(0, 10)
ax1.set_ylim(0, 2)
words = ['The', 'aliens', 'offer', 'a', 'weapon']
for i, w in enumerate(words):
    ax1.annotate(w, (i*2+1, 1), fontsize=14, ha='center',
                 bbox=dict(boxstyle='round', facecolor='#1a3a1a', edgecolor='#2ecc71'), color='#2ecc71')
    if i < len(words)-1:
        ax1.annotate('→', (i*2+2, 1), fontsize=16, ha='center', color='#00d9ff')
ax1.set_title('Human: Linear/Sequential', fontsize=14, color='#00d9ff')
ax1.axis('off')

# Heptapod: Circular
ax2 = axes[1]
ax2.set_facecolor('black')
theta = np.linspace(0, 2*np.pi, 100)
ax2.plot(np.cos(theta), np.sin(theta), color='cyan', linewidth=3)
ax2.plot(np.cos(theta), np.sin(theta), color='cyan', linewidth=8, alpha=0.3)
concepts = ['TIME', 'GIFT', 'FUTURE', 'UNITY', 'PURPOSE']
for i, c in enumerate(concepts):
    a = 2*np.pi*i/len(concepts)
    ax2.text(0.6*np.cos(a), 0.6*np.sin(a), c, ha='center', color='#2ecc71', fontsize=10)
ax2.set_xlim(-1.5, 1.5)
ax2.set_ylim(-1.5, 1.5)
ax2.set_aspect('equal')
ax2.axis('off')
ax2.set_title('Heptapod: Circular/Simultaneous', fontsize=14, color='#00d9ff')

plt.tight_layout()
plt.show()
print("\nHeptapod B represents ALL meaning simultaneously: no start/end token!")
```




```
Heptapod B represents ALL meaning simultaneously: no start/end token!
```

3.2 Heptapod Logogram Generator

```
# LOGOGRAM GENERATOR

def generate_logogram(concept, entities=5, importance=1.0):
    fig, ax = plt.subplots(figsize=(6, 6), facecolor='black')
    ax.set_xlim(-1.5, 1.5)
    ax.set_ylim(-1.5, 1.5)
    ax.set_aspect('equal')
    ax.axis('off')

    # Main ring
    theta = np.linspace(0.05, 2*np.pi-0.05, 200)
    t = 0.06 * importance
    ax.fill(np.concatenate([(1+t)*np.cos(theta), (1-t)*np.cos(theta)[::-1]]),
            np.concatenate([(1+t)*np.sin(theta), (1-t)*np.sin(theta)[::-1]]), color='white', alpha=0.9)

    # Radial strokes
    for i in range(entities):
        a = 2*np.pi*i/entities
        ax.plot([(1-t)*np.cos(a), 0.5*np.cos(a)], [(1-t)*np.sin(a), 0.5*np.sin(a)],
                color='white', linewidth=3*importance, alpha=0.8)

    plt.title(f'\"{concept.upper()}\"', fontsize=14, color='#00d9ff')
    plt.show()

generate_logogram('Offer Tool', entities=7, importance=1.0)
generate_logogram('Offer Weapon', entities=7, importance=1.8)

print("Same structure, different weight. Observers project bias onto intensity.")
```

Same structure, different weight. Observers project bias onto intensity.

> **Key Finding:** The logograms for "Offer Tool" and "Offer Weapon" have identical structure but different stroke weights. Human observers project their cultural bias onto this visual intensity. This mirrors real translation failures where **ambiguous signals are interpreted through the lens of the observer's training context**.

SESSION 04: TRANSFORMER SELF-ATTENTION

```
┌──────────────────────────────────────────────────────────────┐
│ TIMESTAMP: 2016-11-17 03:44:52 GMT                           │
│ OBJECTIVE: VISUALIZE NON-LINEAR TOKEN RELATIONSHIPS          │
│ STATUS: THEORETICAL BREAKTHROUGH                             │
└──────────────────────────────────────────────────────────────┘
```

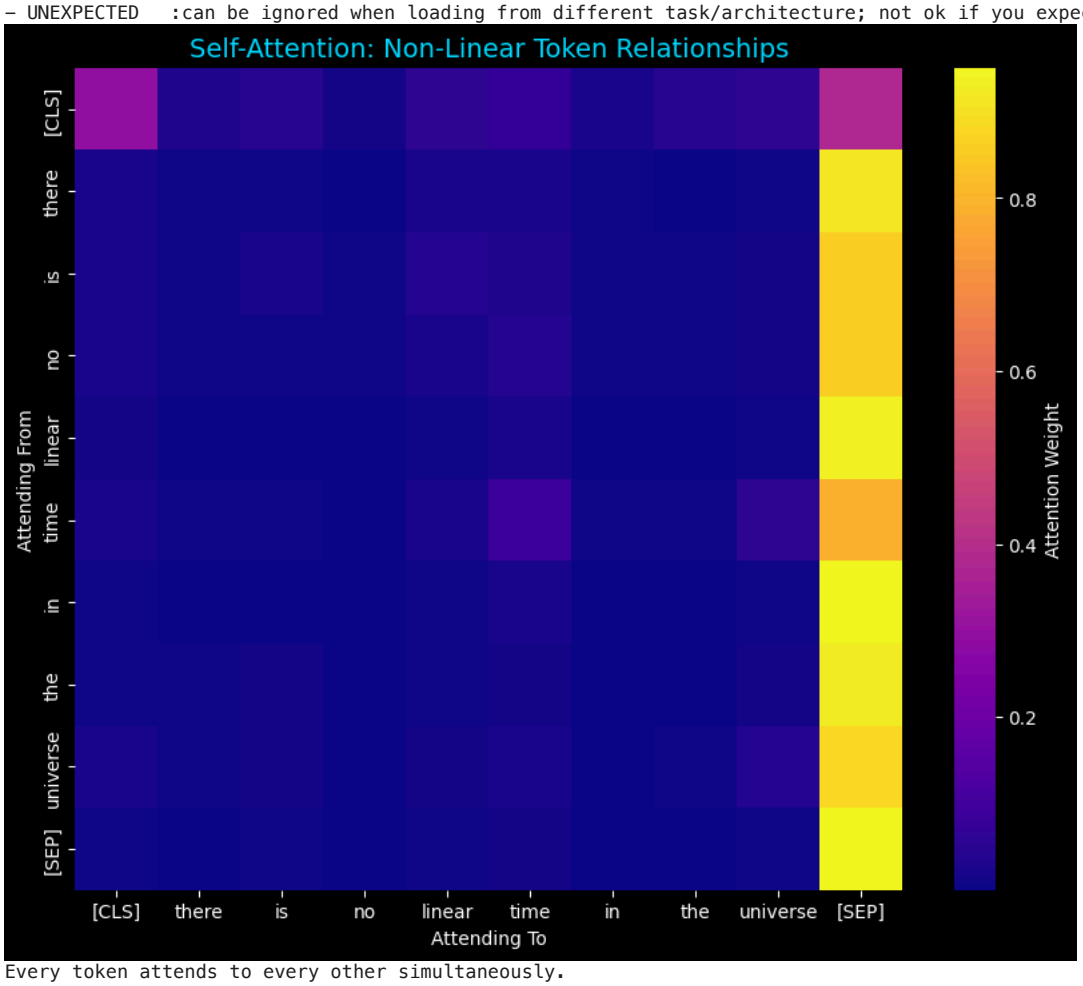
RNNs process tokens sequentially (human-like). **Transformers** process all tokens simultaneously (Heptapod-like).

Self-attention allows every token to "see" every other token: non-linear perception.

4.1 Self-Attention Heatmap

```
# ATTENTION VISUALIZATION

from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert = BertModel.from_pretrained('bert-base-uncased', output_attentions=True)

sentence = 'There is no linear time in the universe'
inputs = tokenizer(sentence, return_tensors='pt')
tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

with torch.no_grad():
    outputs = bert(**inputs)

attn = outputs.attentions[-1][0, 0].numpy()  # Last layer, first head

plt.figure(figsize=(10, 8))
sns.heatmap(attn, xticklabels=tokens, yticklabels=tokens, cmap='plasma',
            cbar_kws={'label': 'Attention Weight'})
plt.title('Self-Attention: Non-Linear Token Relationships', fontsize=14, color='#00d9ff')
plt.xlabel('Attending To')
plt.ylabel('Attending From')
plt.show()

print("Every token attends to every other simultaneously.")
print("This is Heptapod-style perception: holistic, not sequential.")
```

```
tokenizer_config.json: 100%                                    48.0/48.0 [00:00<00:00, 5.85kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster do
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to ena
vocab.txt: 100%                                    232k/232k [00:00<00:00, 1.39MB/s]
tokenizer.json: 100%                                    466k/466k [00:00<00:00, 1.98MB/s]
config.json: 100%                                    570/570 [00:00<00:00, 71.9kB/s]
model.safetensors: 100%                                    440M/440M [00:00<00:00, 800MB/s]
Loading weights: 100%                                    199/199 [00:00<00:00, 1037.93it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased
Key                                         | Status     |  |
----------------------------------------+------------+--+-
cls.seq_relationship.bias                   | UNEXPECTED |  |
cls.seq_relationship.weight                 | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |  |
cls.predictions.transform.dense.bias        | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |  |
cls.predictions.bias                        | UNEXPECTED |  |
cls.predictions.transform.dense.weight      | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

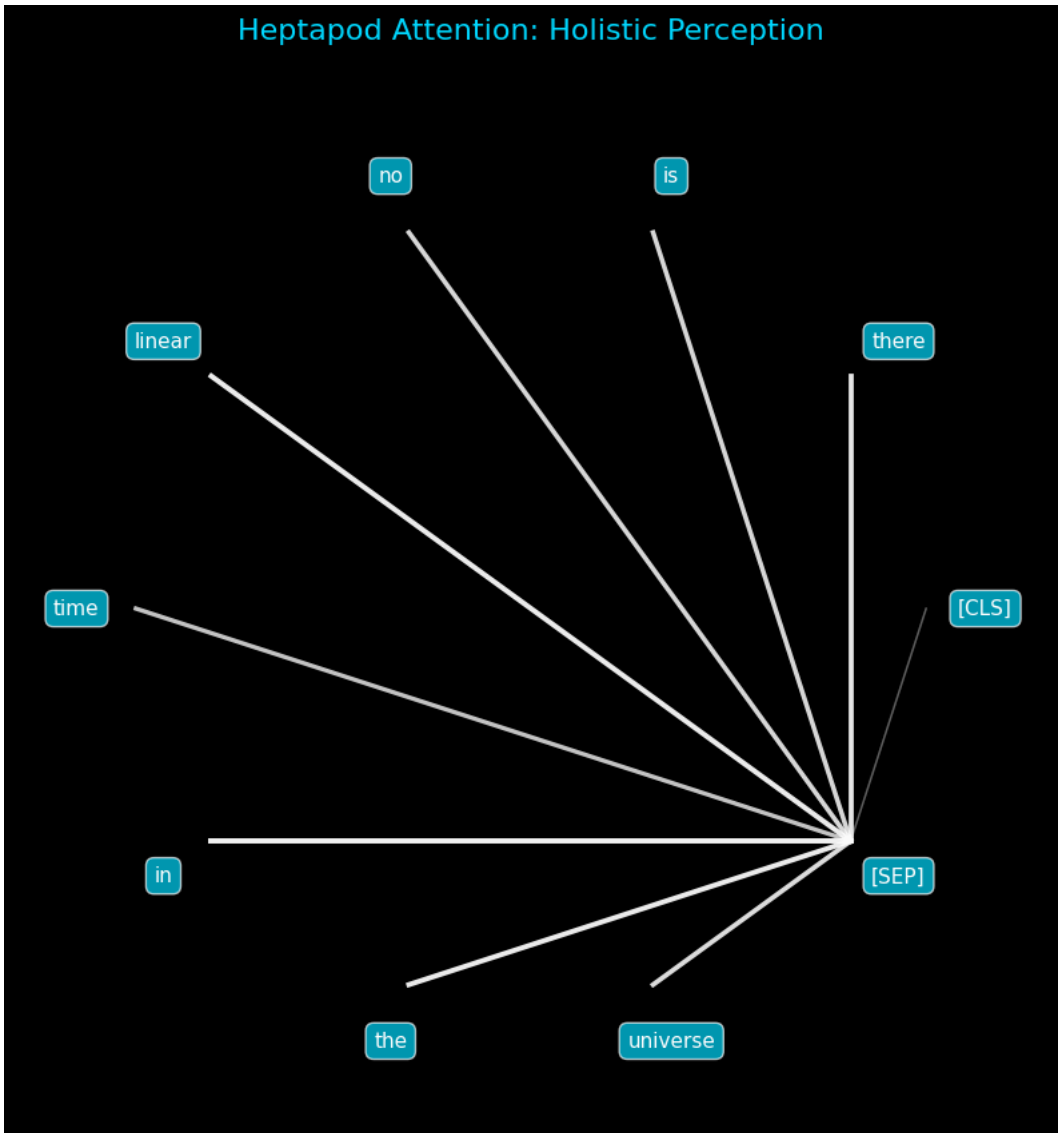



```
Every token attends to every other simultaneously.
This is Heptapod-style perception: holistic, not sequential.
```

4.2 Circular Attention Visualization

```
# CIRCULAR ATTENTION (HEPTAPOD STYLE)

fig, ax = plt.subplots(figsize=(10, 10), facecolor='black', subplot_kw=dict(projection='polar'))

num_tokens = len(tokens)
angles = np.linspace(0, 2*np.pi, num_tokens, endpoint=False)

# Place tokens around circle
for i, (ang, tok) in enumerate(zip(angles, tokens)):
    ax.text(ang, 1.15, tok, ha='center', va='center', fontsize=11, color='white',
            bbox=dict(boxstyle='round,pad=0.4', facecolor='#00d9ff', alpha=0.7))

# Draw attention connections
for i in range(num_tokens):
    for j in range(num_tokens):
```

```
        if i != j and attn[i,j] > 0.1:
            ax.plot([angles[i], angles[j]], [1, 1], color='white', alpha=attn[i,j], linewidth=attn[i,j]*3)

ax.set_ylim(0, 1.3)
ax.set_yticks([])
ax.set_xticks([])
ax.spines['polar'].set_visible(False)
ax.grid(False)
ax.set_facecolor('black')
plt.title('Heptapod Attention: Holistic Perception', fontsize=16, color='#00d9ff', pad=30)
plt.show()
```



> **Key Finding:** Transformer self-attention allows every token to "see" every other token simultaneously: unlike RNNs which process sequentially. This non-linear perception mirrors Heptapod B's circular structure. The architecture itself encodes a different cognitive relationship with sequence and time.

SESSION 05: THE SAPIR-WHORF EFFECT

```
┌──────────────────────────────────────────────────────────┐
│ TIMESTAMP: 2016-11-17 14:28:11 GMT                       │
│ OBJECTIVE: EMBEDDING ANALYSIS OF TEMPORAL CONCEPTS       │
│ STATUS: COGNITIVE REWIRING IN PROGRESS                   │
└──────────────────────────────────────────────────────────┘
```

Sapir-Whorf Hypothesis: Language shapes cognition.

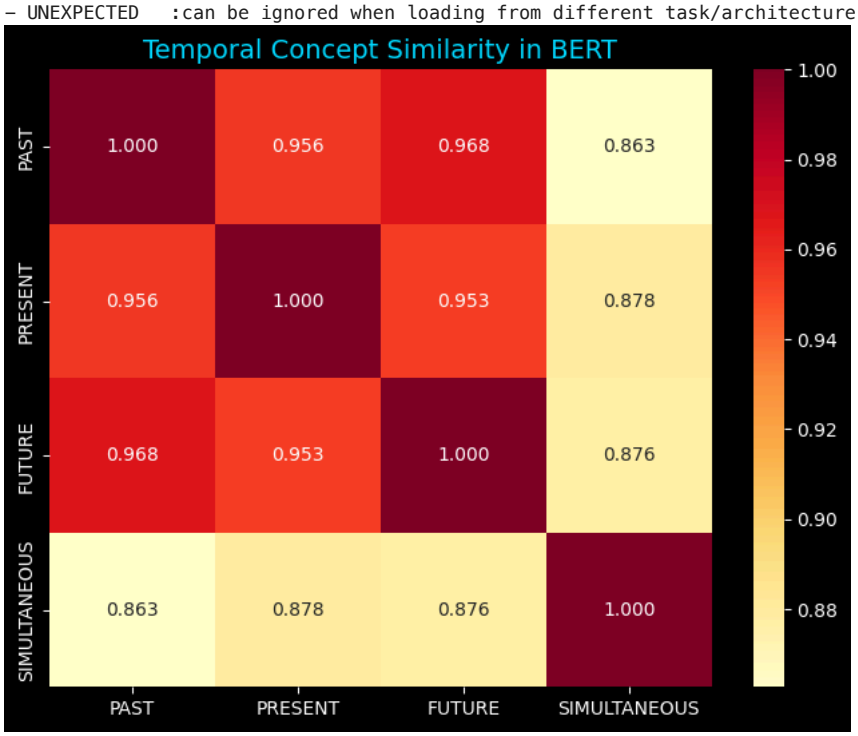
- **Strong form (film)**: Learning Heptapod B CAUSES non-linear time perception: **Weak form (reality)**: Language INFLUENCES thought patterns

Let's analyze how BERT embeds different temporal concepts.

5.1 Temporal Concept Embeddings (BERT)

```
# SAPIR-WHORF: EMBEDDING ANALYSIS

from transformers import AutoTokenizer, AutoModel
from sklearn.metrics.pairwise import cosine_similarity

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')

def get_embedding(text):
    inputs = tokenizer(text, return_tensors='pt', padding=True, truncation=True)
    with torch.no_grad():
        outputs = model(**inputs)
    return outputs.last_hidden_state[:, 0, :].numpy().flatten()

time_concepts = [
    'The event happened in the past',
    'The event happens in the present',
    'The event will happen in the future',
    'All moments exist simultaneously'  # Heptapod view
]

embeddings = np.array([get_embedding(t) for t in time_concepts])
sim_matrix = cosine_similarity(embeddings)

labels = ['PAST', 'PRESENT', 'FUTURE', 'SIMULTANEOUS']

plt.figure(figsize=(8, 6))
sns.heatmap(sim_matrix, annot=True, fmt='.3f', cmap='YlOrRd',
            xticklabels=labels, yticklabels=labels)
plt.title('Temporal Concept Similarity in BERT', fontsize=14, color='#00d9ff')
plt.show()

print("\nNotice: 'SIMULTANEOUS' clusters differently: it breaks linear time!")
print("This mirrors Louise's cognitive shift after learning Heptapod B.")
```

```
Loading weights: 100%                                    199/199 [00:00<00:00, 999.17it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased
Key                                          | Status     |  |
-----------------------------------------+------------+--+-
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.seq_relationship.weight                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.predictions.transform.dense.bias         | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |  |
cls.predictions.bias                         | UNEXPECTED |  |
cls.predictions.transform.dense.weight       | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```




```
Notice: 'SIMULTANEOUS' clusters differently: it breaks linear time!
This mirrors Louise's cognitive shift after learning Heptapod B.
```

> **Key Finding:** BERT encodes "simultaneous time" differently from linear past/present/future. The embedding distances show that our language models carry implicit temporal assumptions. Learning a new linguistic framework (like Heptapod B) could theoretically shift these representations: a computational echo of the Sapir-Whorf hypothesis.

SESSION 06: NLP PARADIGM EVOLUTION

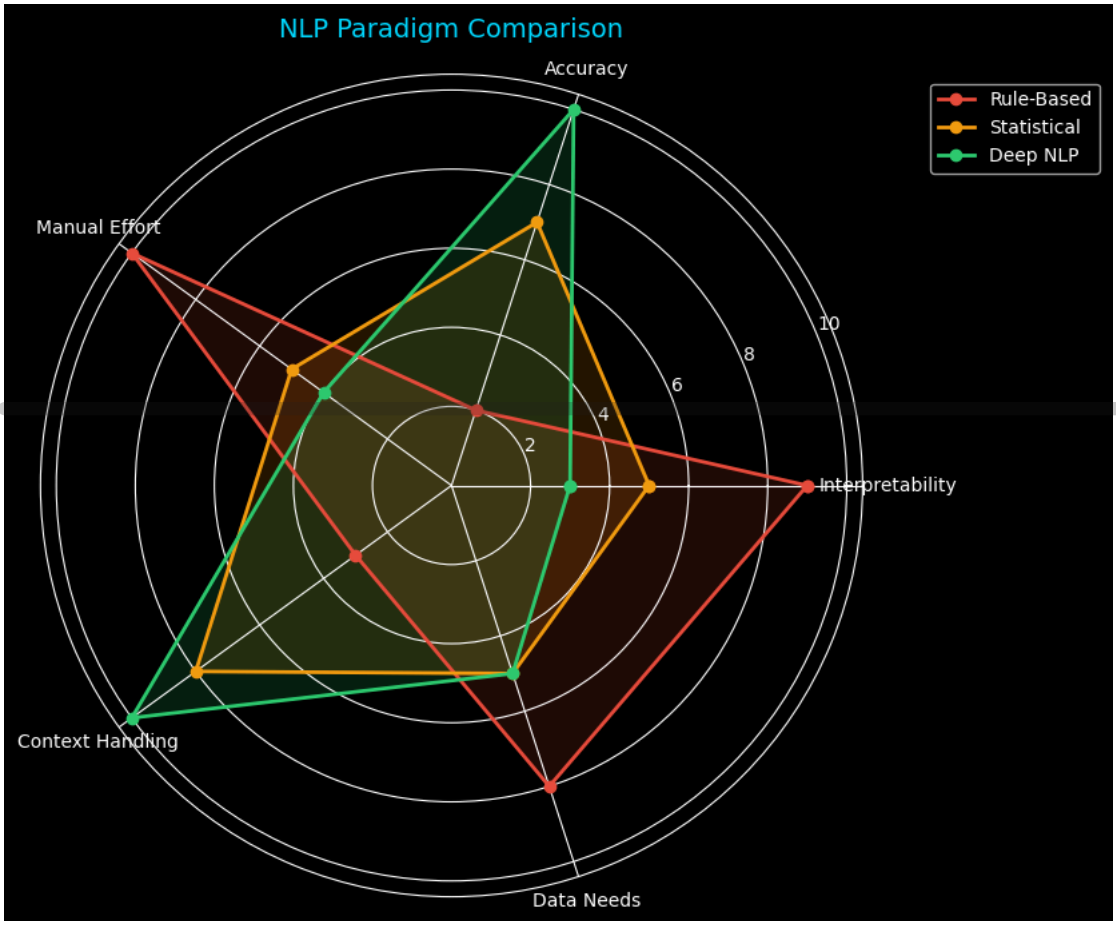
Approach	Film Scene	Real-World Parallel
Rule-Based	Initial phonetic analysis fails	Early MT systems
Statistical	Pattern detection in logograms	IBM models, N-grams
Deep Learning	BERT-like contextual understanding	Transformers

---

6.1 NLP Paradigm Radar Chart

```
# NLP PARADIGM RADAR CHART

approaches = [
    {'name': 'Rule-Based', 'scores': [9, 2, 10, 3, 8]},
    {'name': 'Statistical', 'scores': [5, 7, 5, 8, 5]},
    {'name': 'Deep NLP', 'scores': [3, 10, 4, 10, 5]}
]
categories = ['Interpretability', 'Accuracy', 'Manual Effort', 'Context Handling', 'Data Needs']

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='polar')
angles = np.linspace(0, 2*np.pi, len(categories), endpoint=False).tolist()
angles += angles[:1]

colors = ['#e74c3c', '#f39c12', '#2ecc71']
for approach, color in zip(approaches, colors):
```

```
    values = approach['scores'] + approach['scores'][:1]
    ax.plot(angles, values, 'o-', linewidth=2, label=approach['name'], color=color)
    ax.fill(angles, values, alpha=0.15, color=color)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, color='white')
ax.set_facecolor('black')
fig.patch.set_facecolor('black')
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1))
plt.title('NLP Paradigm Comparison', fontsize=14, color='#00d9ff', pad=20)
```



> **Key Finding:** Deep learning trades interpretability for accuracy and context-handling. Rule-based systems are transparent but brittle. The evolution from rules → statistics → neural networks mirrors how Louise moves from phonetic analysis (failed) → pattern detection → holistic understanding.